

Week -1

Write a program to Create an avl tree and perform insertion, deletion operations on it?

```
#include <stdio.h>
#include <stdlib.h>

// AVL Tree Node
struct Node {
    int key;
    int height;
    struct Node* left;
    struct Node* right;
};

// Create a new node
struct Node* createNode(int key) {
    struct Node* node = (struct Node*)malloc(sizeof(struct Node));
    node->key = key;
    node->left = node->right = NULL;
    node->height = 1; // new node is initially added at leaf
    return node;
}

// Get height of node
int height(struct Node* node) {
    return node ? node->height : 0;
}

// Get max of two integers
int max(int a, int b) {
    return (a > b) ? a : b;
}

// Update height of node
void updateHeight(struct Node* node) {
    node->height = 1 + max(height(node->left), height(node->right));
}

// Get balance factor of node
int getBalance(struct Node* node) {
    return node ? height(node->left) - height(node->right) : 0;
}

// Right rotate
struct Node* rightRotate(struct Node* y) {
    struct Node* x = y->left;
    struct Node* T2 = x->right;

    x->right = y;
    y->left = T2;

    updateHeight(y);
    updateHeight(x);

    return x;
}
```

```

// Left rotate
struct Node* leftRotate(struct Node* x) {
    struct Node* y = x->right;
    struct Node* T2 = y->left;

    y->left = x;
    x->right = T2;

    updateHeight(x);
    updateHeight(y);

    return y;
}

// Insert a key into the AVL Tree
struct Node* insert(struct Node* node, int key) {
    // Perform normal BST insert
    if (node == NULL)
        return createNode(key);

    if (key < node->key)
        node->left = insert(node->left, key);
    else if (key > node->key)
        node->right = insert(node->right, key);
    else
        return node; // No duplicate keys

    // Update height and balance the node
    updateHeight(node);

    int balance = getBalance(node);

    // Balance the tree using rotations
    // Case 1 - Left Left
    if (balance > 1 && key < node->left->key)
        return rightRotate(node);

    // Case 2 - Right Right
    if (balance < -1 && key > node->right->key)
        return leftRotate(node);

    // Case 3 - Left Right
    if (balance > 1 && key > node->left->key) {
        node->left = leftRotate(node->left);
        return rightRotate(node);
    }

    // Case 4 - Right Left
    if (balance < -1 && key < node->right->key) {
        node->right = rightRotate(node->right);
        return leftRotate(node);
    }

    return node;
}

// In-order traversal

```

```

void inorder(struct Node* root) {
    if (root != NULL) {
        inorder(root->left);
        printf("%d ", root->key);
        inorder(root->right);
    }
}

// Main program
int main() {
    struct Node* root = NULL;

    // Insert elements
    root = insert(root, 50);
    root = insert(root, 30);
    root = insert(root, 70);
    root = insert(root, 20);
    root = insert(root, 40);
    root = insert(root, 60);
    root = insert(root, 80);

    printf("In-order Traversal of AVL Tree:\n");
    inorder(root);
    printf("\n");

    return 0;
}

```

Program Output:
 In-order Traversal of AVL Tree:
 20 30 40 50 60 70 80

```

#include <stdio.h>
#include <stdlib.h>

// AVL Tree Node
struct Node {
    int key;
    int height;
    struct Node* left;
    struct Node* right;
};

// Create a new node

```

```

struct Node* createNode(int key) {
    struct Node* node = (struct Node*)malloc(sizeof(struct Node));
    node->key = key;
    node->left = node->right = NULL;
    node->height = 1;
    return node;
}

int height(struct Node* node) {
    return node ? node->height : 0;
}

int max(int a, int b) {
    return (a > b) ? a : b;
}

void updateHeight(struct Node* node) {
    node->height = 1 + max(height(node->left), height(node->right));
}

int getBalance(struct Node* node) {
    return node ? height(node->left) - height(node->right) : 0;
}

// Right rotate
struct Node* rightRotate(struct Node* y) {
    struct Node* x = y->left;
    struct Node* T2 = x->right;

    x->right = y;
    y->left = T2;

    updateHeight(y);
    updateHeight(x);

    return x;
}

// Left rotate
struct Node* leftRotate(struct Node* x) {
    struct Node* y = x->right;
    struct Node* T2 = y->left;

    y->left = x;
    x->right = T2;

    updateHeight(x);
    updateHeight(y);

    return y;
}

// Insert a key
struct Node* insert(struct Node* node, int key) {
    if (node == NULL)
        return createNode(key);

    if (key < node->key)

```

```

        node->left = insert(node->left, key);
    else if (key > node->key)
        node->right = insert(node->right, key);
    else
        return node; // No duplicates

    updateHeight(node);
    int balance = getBalance(node);

    // Rebalance if needed
    if (balance > 1 && key < node->left->key)
        return rightRotate(node);
    if (balance < -1 && key > node->right->key)
        return leftRotate(node);
    if (balance > 1 && key > node->left->key) {
        node->left = leftRotate(node->left);
        return rightRotate(node);
    }
    if (balance < -1 && key < node->right->key) {
        node->right = rightRotate(node->right);
        return leftRotate(node);
    }

    return node;
}

// Find node with minimum key value
struct Node* minValueNode(struct Node* node) {
    struct Node* current = node;
    while (current->left != NULL)
        current = current->left;
    return current;
}

// Delete a node from AVL tree
struct Node* deleteNode(struct Node* root, int key) {
    if (root == NULL)
        return root;

    // Perform BST delete
    if (key < root->key)
        root->left = deleteNode(root->left, key);
    else if (key > root->key)
        root->right = deleteNode(root->right, key);
    else {
        // Node with one or no child
        if ((root->left == NULL) || (root->right == NULL)) {
            struct Node* temp = root->left ? root->left : root->right;
            if (temp == NULL) {
                temp = root;
                root = NULL;
            } else {
                *root = *temp;
            }
            free(temp);
        } else {
            // Node with two children
            struct Node* temp = minValueNode(root->right);

```

```

        root->key = temp->key;
        root->right = deleteNode(root->right, temp->key);
    }
}

if (root == NULL)
    return root;

updateHeight(root);
int balance = getBalance(root);

// Rebalance
if (balance > 1 && getBalance(root->left) >= 0)
    return rightRotate(root);
if (balance > 1 && getBalance(root->left) < 0) {
    root->left = leftRotate(root->left);
    return rightRotate(root);
}
if (balance < -1 && getBalance(root->right) <= 0)
    return leftRotate(root);
if (balance < -1 && getBalance(root->right) > 0) {
    root->right = rightRotate(root->right);
    return leftRotate(root);
}

return root;
}

// In-order traversal
void inorder(struct Node* root) {
    if (root != NULL) {
        inorder(root->left);
        printf("%d ", root->key);
        inorder(root->right);
    }
}

// Main function
int main() {
    struct Node* root = NULL;

    // Insert elements
    root = insert(root, 50);
    root = insert(root, 30);
    root = insert(root, 70);
    root = insert(root, 20);
    root = insert(root, 40);
    root = insert(root, 60);
    root = insert(root, 80);

    printf("In-order traversal before deletion:\n");
    inorder(root);
    printf("\n");

    // Delete a node
    root = deleteNode(root, 30);

    printf("In-order traversal after deleting 30:\n");

```

```
    inorder(root);  
    printf("\n");  
  
    return 0;  
}
```

Program Output:

In-order traversal before deletion:

20 30 40 50 60 70 80

In-order traversal after deleting 30:

20 40 50 60 70 80